LAB 5: STATIC CODE ANALYSIS

Student: Abhirama H Y | SRN: PES2UG24CS023 | Section: A

1. Objective

To evaluate a Python scholarship-disbursement program using Pylint, Bandit, and Flake8; document issues; apply fixes; and verify the changes with before-and-after reports.

2. Tools and Environment

Environment: Windows PowerShell, Python 3.13.9, and a project virtual environment (.venv). Installed versions reported by the terminal: Pylint 4.1.3, Bandit 1.9.4, and Flake8 7.4.1.

3. Issues Identified and Fixes

Issue	Type / Tool	Before line(s)	Fix applied	Verification / result
SQL string interpolation	Security / Bandit	scholarship_disbursement_original.py:10	Replaced string formatting/interpolation with a parameterized SQL statement.	BEFORE: Bandit B608, Medium severity, Medium confidence. AFTER: no issues identified.
Unsafe shell execution	Security / Bandit	scholarship_disbursement_original.py:26	Removed os.system shell execution; use an appropriate Python API or remove the unnecessary operation.	BEFORE: Bandit B605 and B607, Low severity. AFTER: no issues identified.
Swallowed exception (try/except/pass)	Security / Bandit; quality / Pylint	scholarship_disbursement_original.py:27–28	Removed the silent pass; handle expected exceptions explicitly and log/re-raise when appropriate.	BEFORE: Bandit B110 (Low); Pylint W0718 at line 27 for catching Exception. AFTER Bandit: no issues identified.
Mutable default list	Quality / Pylint	scholarship_disbursement_original.py:9	Use audit=None and initialize a fresh list inside the function.	BEFORE: Pylint W0102 (dangerous-default-value). The AFTER Pylint report does not list this finding.
Non-PEP 8 function naming	Quality / Pylint	scholarship_disbursement_original.py:31	Rename bySchemeCount to count_by_scheme and update its call sites.	BEFORE: Pylint C0103 (invalid-name). The AFTER Pylint report does not list this finding.
Incorrect None comparison	Quality / Pylint and Flake8	scholarship_disbursement_original.py:32	Use `scheme is None` rather than `scheme == None`.	BEFORE: Pylint C0121 and Flake8 E711. Neither finding appears in the supplied AFTER reports.
Unused import	Quality / Pylint and Flake8	Not present in original report; appears in AFTER at scholarship_disbursement.py:18	Remove the unused `Iterable` import from typing.	STILL PRESENT AFTER: Pylint W0611; Flake8 F401.
Line-length/style violations	Style / Pylint and Flake8	Original Flake8: lines 5, 10, 17, 19, 34; line 32 also has E711	Wrap long lines and follow the configured line-length/style rules.	STILL PRESENT AFTER Flake8: E501 at lines 104, 153, and 154. Final Flake8 report is not

				clean.
Missing module/function docstrings and other quality findings	Quality / Pylint	Original Pylint: module line 1; functions at lines 9, 15, 24, 31, 37; other findings at lines 6, 15–16	Add docstrings, use a consistent return strategy, avoid global state, and follow naming conventions.	BEFORE Pylint score: 5.00/10. AFTER Pylint score: 9.86/10, with one remaining unused-import warning.
Input validation	Correctness / manual inspection	Not reported by the supplied static-analysis reports	Validate non-empty identifiers/scheme and require a finite positive amount.	Manual-review item; no separate test output was provided, so do not claim test verification from these reports.
Duplicate disbursement handling	Data integrity / manual inspection	Not reported by the supplied static-analysis reports	Prevent duplicate student/scheme records with a database constraint and handle integrity errors.	Manual-review item; no separate duplicate-test output was provided, so do not claim test verification from these reports.

4. Analyzer Results Summary

Bandit: BEFORE: 4 findings — one Medium (B608 SQL construction) and three Low (B605, B607, B110). AFTER: “No issues identified” (0 Low, 0 Medium, 0 High).

Pylint: BEFORE score: 5.00/10 with multiple code-quality findings. AFTER score: 9.86/10; the report still identifies W0611 for the unused Iterable import at line 18.

Flake8: BEFORE: 6 reported issues (five E501 long-line findings and one E711 None comparison). AFTER: 4 reported issues — F401 unused Iterable at line 18 and E501 at lines 104, 153, and 154.

5. Reflection

Q1. What is static code analysis, and how is it useful?

Static code analysis examines source code without running the program. It helps identify likely bugs, insecure patterns, and style violations early. It complements tests and runtime checks.

Q2. What differences did you observe between Pylint, Bandit, and Flake8?

Pylint focuses broadly on code quality, naming, design, and possible logical problems and provides a score. Bandit focuses on security-sensitive patterns. Flake8 primarily checks style and basic issues such as whitespace, line length, and unused names.

Q3. Which issues were most important, and why?

Security issues such as unsafe SQL construction or shell execution deserve high priority because they may enable unintended operations. Duplicate disbursements and hidden exceptions also matter because they can damage data integrity or conceal failures.

Q4. How did static analysis improve the program, and what are its limitations?

The review encouraged safer SQL, explicit validation, safer exception handling, clearer naming, and consistent formatting. A clean report does not prove correctness or security; tools can miss context-dependent defects or produce false positives, so tests and manual review remain necessary.